AIA1: Advanced AI Architectures

Introduction and Theoretical Foundations

Advanced AI Architectures form the backbone of modern generative AI systems, enabling models to process sequences, generate content, and scale to trillions of parameters. Theoretically, these architectures address fundamental limitations of prior paradigms like RNNs (sequential processing bottlenecks) and CNNs (limited receptive fields). Drawing from universal approximation theorems and scaling laws (Kaplan et al., 2020), they achieve emergent capabilities through massive parameterization and data exposure.

This module covers four pillars:

- **Transformers and Attention**: Parallelizable sequence modeling.
- **Diffusion Models**: Iterative denoising for generation.
- **Mixture-of-Experts (MoE)**: Sparse activation for efficiency.

Theoretical Motivation: Information bottleneck theory (Tishby 1999) explains why attention dynamically compresses inputs. Kolmogorov-Arnold representation theorem underpins MoE's expert composition.

Historical Context: Transformers (Vaswani 2017) revolutionized NLP; diffusion (Sohl-Dickstein 2015) revived probabilistic generation; MoE (Jacobs 1991) evolved for deep learning.

(Figure 1: Evolution timeline from RNNs → Transformers → Diffusion/MoE)

Transformers: Theory and Self-Attention Mechanism

Transformers model sequences via **self-attention**, theoretically capturing all pairwise interactions in $O(n^2)$ time—optimal for dependency modeling per permutation-equivariance.

Scaled Dot-Product Attention derives from kernel methods:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}} + B\right)V$$

- $Q = XW_Q, K = XW_K, V = XW_V$ ($X \in \mathbb{R}^{n \times d}$).
- Softmax kernel approximates Gaussian RBF for similarity.
- **Proof of Scaling**: Without $\sqrt{d_k}$, dot products have variance d_k, causing softmax gradient vanishing (variance → ∞).

Multi-Head Attention: Ensemble of h projections, theoretically diversifying subspaces (syntactic vs. semantic heads observed empirically).

Positional Encoding Theory: Absolute (sin/cos) injects translation equivariance; Rotary Position Embeddings (RoPE, Su 2021) preserve relative distances:

$$q_m^{(i)} \leftarrow q_m^{(i)} \cos(m\theta_i) + q_m^{(i+1)} \sin(m\theta_i)$$

Theorem: RoPE enables length extrapolation up to 4x training length.

Full Transformer Block:

1. Self-attn → Add & Norm.
2. FFN (MLP) → Add & Norm.
   Residuals ensure gradient flow (identity mapping theorem).

Complexity: Time $O(Ln^2d)$, Space $O(n^2)$ → Linear attn approximations (Reformer) use hashing.

(Figure 2: Attention matrix heatmap for sentence dependency)

Transformer Variants and Scaling Theory

Decoder-Only (GPT-style): Causal mask $M_{ij} = -\infty$ for $i < j$, autoregressive $p(x_t \mid x_{<t})$.

Encoder-Only (BERT): Bidirectional, [MASK] pretraining.

Scaling Laws: Loss $L(N, D, C) \approx a/N^{\alpha} + b/D^{\beta} + c$, optimal $N \propto C^{0.5}$ (Hoffmann Chinchilla).

Extended Theory: Grokking (Power 2022)—sudden generalization post-overfitting, linked to phase transitions in loss landscape.

Practical Implementation Theory: FlashAttention fuses softmax+matmul, reducing I/O via tiling (Dao 2022, 3x speedup).

Code: Minimal Transformer (PyTorch):

python

```
import torch, torch.nn as nn

class SelfAttention(nn.Module):

  def __init__(self, d_model, n_heads):

    self.scale = d_model ** -0.5

    self.qkv = nn.Linear(d_model, 3*d_model)

    self.proj = nn.Linear(d_model, d_model)

    self.n_heads = n_heads

  def forward(self, x):  # x: (B, T, D)

    B,T,D = x.shape

    qkv = self.qkv(x).reshape(B,T,3,self.n_heads,D//self.n_heads).permute(2,0,1,3,4)

    q,k,v = qkv[0],qkv[1],qkv[2]

    attn = (q @ k.transpose(-2,-1)) * self.scale

    mask = torch.triu(torch.ones(T,T), diagonal=1).bool().to(x.device)

    attn.masked_fill_(mask, float('-inf'))
```

```
    attn = attn.softmax(-1)
    out = attn @ v
    return self.proj(out.transpose(1,2).contiguous().view(B,T,D))
```

Diffusion Models: Probabilistic Generation Theory

Diffusion models theoretically reverse **Markov diffusion processes**, sampling from $p(x)$ via learned reverse SDE.

Forward Diffusion (variance-preserving):

$$q(x_t \mid x_{t-1}) = \mathcal{N}(x_t; \sqrt{1-\beta_t} x_{t-1}, \beta_t I)$$

Closed-form: $x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1-\bar{\alpha}_t}\epsilon$, $\epsilon \sim \mathcal{N}(0, I)$.

Reverse (DDPM, Ho 2020): $p_\theta(x_{t-1} \mid x_t) = \mathcal{N}(\mu_\theta, \tilde{\beta}_t I)$.

Training Objective (variational lower bound simplification):

$$L_t^{\text{simple}} = \mathbb{E}_{\epsilon,t} \parallel \epsilon - \epsilon_\theta(\sqrt{\bar{\alpha}_t} x_0 + \sqrt{1-\bar{\alpha}_t}\epsilon, t) \parallel^2$$

Theoretical Justification: Equivalent to denoising score matching ($\nabla \log\ p_t(x_t)$).

Schedules: Cosine (Nichol 2021) > linear for FID scores.

Latent Diffusion (LDM, Rombach 2022): Diffuse in VQ-VAE latents, condition via cross-attn:

$$\epsilon_\theta(z_t, t, c)(c = \text{text embeds})$$

(Figure 3: Denoising trajectory—noise → image over 50 steps)

SDE Formulation (Song 2021): Continuous-time, enables distillation (consistency models).

Diffusion Architectures and Extensions

U-Net Backbone: Encoder-decoder with skip connections, time embeds via FiLM.

Classifier-Free Guidance: $\tilde{\epsilon} = \epsilon_\theta(z_t, t) + s(\epsilon_\theta(z_t, t, c) - \epsilon_\theta(z_t, t \mid \emptyset))$.

Theoretical Advantages over GANs:

- Mode coverage (no collapse).
- ELBO maximization.

Code Snippet: DDPM Step:

python

```
def denoise_step(model, z_t, t, guidance=1.0):
```

```
    pred_noise = model(z_t, t)
    if conditioned:
        pred_noise_uncond = model(z_t, t, empty_prompt)
        pred_noise = pred_noise_uncond + guidance * (pred_noise - pred_noise_uncond)
    alpha_t = alphas[t]
    z_{t-1} = (z_t - (1-alpha_t).sqrt() * pred_noise) / alpha_t.sqrt()
    return z_{t-1}
```

Extensions:

- Video diffusion (Sora).
- Flow-matching (Lipman 2023, ODE paths).

Mixture-of-Experts (MoE): Sparse Scaling Theory

MoE decomposes functions as mixtures: $f(x) = \sum_e g_e(x) E_e(x)$, g_e: gating.

Sparse MoE Layer (Shazeer 2017):

- Router: $g_e(x) = \text{topk}(\text{softmax}(xW_g), k = 2)$.
- Experts: FFNs, only top-k activated (capacity factor 1.25).

Load Balancing Loss:

$$L_{balance} = \alpha N \sum_e f_e \left(\frac{\sum_i F_i \, g_{e,i}}{\sum_i F_i} \right)^2$$

Prevents collapse to few experts.

Theoretical Scaling: Activate E/N params, $O(E)$ speedup vs. dense.

Switch Transformers (Fedus 2021): 1.6T params, top-1 routing.

(Table 1: MoE vs Dense)

Model	**Total Params**	**Active**	**Speedup**
Dense 1T	1T	1T	1x
MoE 1T	1T	12B	8x

Grok-1: 314B MoE, custom router.

Integration, Challenges, Case Studies

Unified Models: DiT (Peebles 2023)—Transformer replaces U-Net in diffusion.

Challenges & Theory:

- Attention quadratic: Longformer/Sparse Transformers.
- Diffusion steps: Progressive (32→1).
- MoE routing noise: Noisy top-k.

Case Studies:

1. **GPT-4**: Transformer + MoE rumors.
2. **Stable Diffusion 3**: Multimodal diffusion.
3. **Mixtral 8x22B**: MoE SOTA open model.

Loss Landscapes: Mode connectivity theory enables MoE merging.

Hands-On, Exercises, and Theoretical Proofs

Project: Build nano-diffusion + Transformer classifier.

Exercises:

1. **Prove**: Attention is permutation matrix for one-hot.
2. **Derive**: Diffusion variance schedule optimality.
3. **Implement**: MoE with JAX.

Proof: Attention Variance:
$\mathbb{E}[(q^T k)^2] = d_k \Rightarrow$ divide by $\sqrt{d_k}$.

Quiz (Advanced):

- RoPE extrapolation limit?
- DDPM equivalence to score matching.